

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192412015
Name	K.S.Dharshana

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

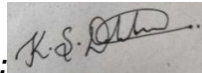
Total Marks:50

Code of Conduct:

IK.S.Dharshana(192412015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

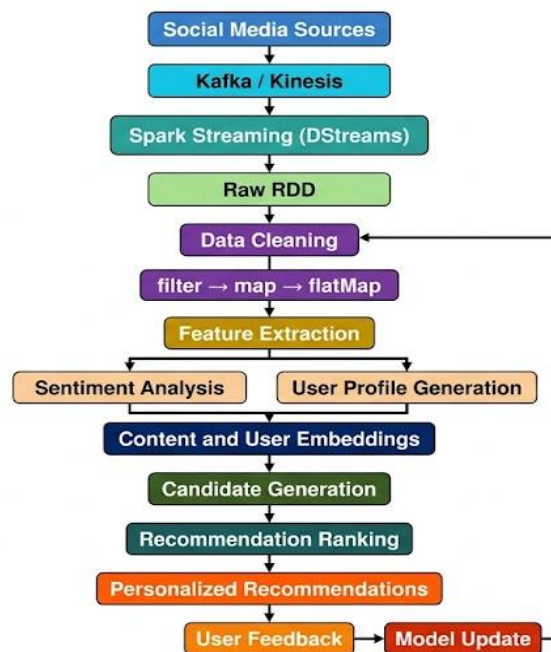
SYSTEM 1 – END-TO-END REAL-TIME SOCIAL MEDIA RECOMMENDATION SYSTEM

1. Introduction

A social media platform receives a huge amount of data every second in the form of posts, comments, likes, shares, follows, searches, and clicks. Processing all this information on a single machine is difficult and slow. A distributed AI system can solve this problem by dividing the data among multiple machines and processing it in parallel.

This system uses **PySpark, RDD operations, Kafka/Kinesis, DStreams, machine-learning models, and intelligent agents** to process social media information in real time and generate personalized recommendations.

2. System Architecture



3. Data Ingestion Layer

Social media events are collected through **Apache Kafka or Amazon Kinesis**. Kafka divides incoming events into partitions, allowing multiple Spark workers to process them at the same time.

A typical event contains:

User ID, Post ID, Event Type, Timestamp, Post Text, Hashtags, Interaction Information

Kafka is useful because it can store incoming events temporarily and replay them if a processing failure occurs.

4. Stream Processing Using DStreams

PySpark DStreams divide the incoming stream into small time-based batches. Each batch is represented as an RDD.

The basic processing sequence is:

Incoming Stream



DStream



RDD



filter()



map()



flatMap()



reduceByKey()



Feature Extraction

filter() removes invalid records.

map() converts the raw record into a standard format.

flatMap() can extract multiple words, hashtags, or topics from a single post.

Example:

```
cleaned = events.filter(is_valid)
```

```
normalized = cleaned.map(normalize_event)
```

```
words = normalized.flatMap(extract_words)
```

5. Sentiment Analysis

The cleaned post text is passed to a sentiment-analysis model. Each post can be classified as positive, negative, or neutral.

For example:

(User101, Positive)

(User102, Neutral)

(User103, Negative)

The sentiment information is useful for understanding the type of content that a user interacts with.

Sentiment information can be aggregated using reduceByKey():

```
sentiment = posts.map(  
    lambda x: (x.user_id, x.sentiment)  
)
```

The aggregated result can become part of the user's profile.

6. User Profile Generation

The system collects information about each user's interests and activities.

For example:

```
user_topics = posts.map(  
    lambda x: (x.user_id, x.topic)  
)
```

The topics can be grouped using:

```
profiles = user_topics.groupByKey()
```

For large datasets, `reduceByKey()` or `aggregateByKey()` is generally preferred when possible because they reduce unnecessary data transfer during the shuffle stage.

A user profile may contain:

User ID

Favourite Topics

Frequently Viewed Content

Interaction Count

Preferred Content Type

Sentiment Preference

Recent Activity

7. Content Embedding

The text of every post is converted into a numerical vector using an embedding model.

Post Text

↓

Cleaning

↓

Tokenization

↓

Embedding Model

↓

Numerical Vector

Content with similar meanings will have similar vectors. These embeddings help the recommendation system identify related content even when the exact words are different.

8. Recommendation Generation

The system first creates a set of possible recommendations. Candidates can come from:

- Similar posts
- Followed accounts
- Trending topics
- Previous interactions

- Collaborative filtering
- Similar content embeddings

User and content features are combined using RDD operations such as join().

```
features = candidates.join(user_features)
```

```
features = features.join(content_features)
```

A ranking model then assigns a score to every candidate.

The score can depend on:

User interest

Content similarity

Recent activity

Predicted engagement

Content freshness

Previous feedback

The highest-scoring content is finally recommended to the user.

9. Intelligent Agent Integration

Intelligent agents can make the recommendation system more adaptive.

Recommendation Monitoring Agent

This agent continuously checks:

- Click-through rate
- Engagement
- Recommendation relevance
- Diversity
- Negative feedback

If recommendation quality decreases, the agent reports the problem and suggests an adjustment.

Ranking Optimization Agent

This agent compares different ranking strategies and identifies which one gives better results.

Current Ranking

↓

Performance Analysis

↓

Alternative Ranking

↓

Testing

↓

Approved Update

The agent should not directly deploy untested changes.

Cold-Start Agent

New users have little or no history. The cold-start agent initially recommends content using:

- Trending topics
- General categories
- Initial user preferences
- Recent interactions

As more user data becomes available, the system gradually creates a personalized profile.

10. Feedback Loop

User actions are continuously collected.

Recommendation



User Action



Click / Like / Share / Skip



Kafka Feedback Topic



Feedback RDD



Model Evaluation



Model Update



Improved Recommendation

This allows the system to learn from actual user behaviour.

11. Fault Tolerance

The system uses several fault-tolerance mechanisms.

1. Kafka stores events so that they can be replayed.
2. RDD lineage allows lost partitions to be recomputed.
3. Important RDDs can be checkpointed to HDFS or cloud storage.
4. Failed Spark tasks can be retried.
5. Multiple Kafka brokers prevent a single broker from becoming a complete failure point.
6. Previous model versions are maintained for rollback.

12. RDD Persistence

Frequently reused RDDs should be cached.

Suitable choices include:

MEMORY_ONLY

MEMORY_AND_DISK

MEMORY_AND_DISK_SER

For large feature or embedding RDDs, MEMORY_AND_DISK_SER can reduce memory usage. If an RDD is too large for memory, MEMORY_AND_DISK is safer.

Caching should only be applied to RDDs that are reused because unnecessary caching wastes memory.

13. Scalability

The system can be scaled by:

- Increasing Kafka partitions.
- Increasing Spark executors.
- Increasing RDD partitions.
- Using dynamic resource allocation.
- Using broadcast variables for small reference data.
- Reducing unnecessary shuffle operations.
- Separating recommendation serving from Spark processing.

A suitable initial benchmark target is:

Input throughput : $\geq 100,000$ events/second

P95 processing latency : ≤ 2 seconds

P99 latency : ≤ 5 seconds

Recommendation P95 : ≤ 200 ms

Availability : $\geq 99.9\%$

14. Serialization

Custom agent classes and feature objects need to be serializable because Spark sends them from the driver to executors.

Large objects should not be repeatedly serialized. Small read-only models and configurations can be sent using broadcast variables.

Agent objects should avoid storing active database connections, network sockets, or file handles.

15. Monitoring and Observability

The system should monitor:

- Kafka throughput
- Spark job duration
- Executor failures
- CPU and memory usage
- Shuffle data
- RDD storage
- Recommendation latency
- Recommendation CTR
- Model accuracy
- Agent decisions

Prometheus and Grafana can be used for metrics and dashboards.

16. Security

All communication between users, Kafka, Spark, storage, and recommendation services should use encrypted connections such as TLS.

Data stored in HDFS or cloud storage should be encrypted at rest.

Role-based access control should be used so that only authorized services can access user data.

17. Conclusion

The proposed system combines real-time streaming, RDD-based processing, AI models, and intelligent agents to provide personalized social media recommendations. Kafka provides reliable data ingestion, while PySpark performs distributed processing. Sentiment analysis, user profiling, and content embeddings improve recommendation quality. Intelligent agents monitor the system, handle ranking optimization, and help solve cold-start problems. RDD lineage, checkpointing, persistence, and scalable Spark resources make the system suitable for large-scale deployment.

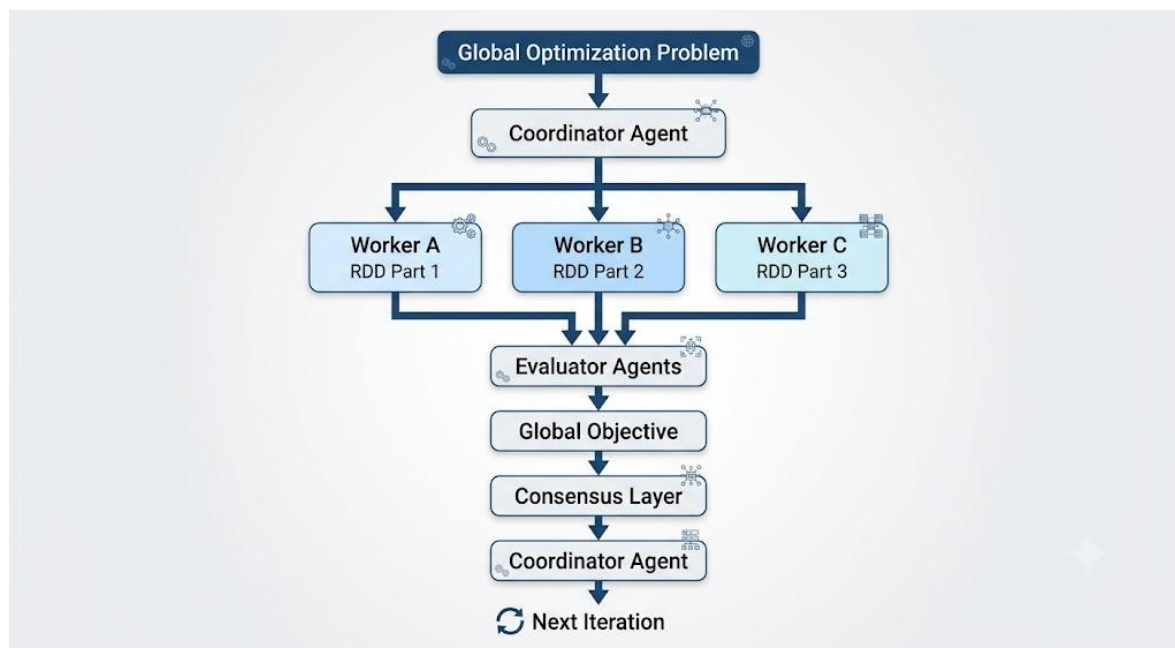
SYSTEM 2 – MULTI-AGENT DISTRIBUTED OPTIMIZATION SYSTEM

1. Introduction

Large optimization problems such as supply-chain routing, portfolio optimization, vehicle scheduling, and resource allocation may contain millions of possible solutions. Solving the entire problem on one machine can take a long time.

A distributed multi-agent system can divide the problem into smaller sub-problems. PySpark processes these sub-problems in parallel using RDD partitions, while intelligent agents cooperate to obtain a good global solution.

2. System Architecture



3. Agent Types

Coordinator Agent

The coordinator divides the overall problem into smaller tasks and assigns them to worker agents. It also manages iterations and combines the results.

Worker Agents

Worker agents solve individual sub-problems. For example, in supply-chain optimization, each worker may handle one geographical region.

Evaluator Agents

Evaluator agents compare the solutions generated by workers using objective values and constraint satisfaction.

Safety or Constraint Agent

This agent checks whether a proposed solution violates any hard constraint.

4. RDD Partitioning

The optimization data can be divided into multiple RDD partitions.

```
problem_rdd = sc.parallelize(problem, 100)
```

The partitions can be assigned logically to different worker agents:

Partition 1 → Worker Agent 1

Partition 2 → Worker Agent 2

Partition 3 → Worker Agent 3

...

Partition 100 → Worker Agent 100

The actual number of active workers can be different from the number of partitions because Spark schedules partitions dynamically.

5. Worker Processing

`mapPartitions()` is useful because each worker can process an entire partition at once.

```
optimized = problem_rdd.mapPartitions(  
    optimize_partition  
)
```

Each worker agent can maintain:

Agent ID

Current Solution

Objective Value

Iteration Number

Constraint Status

Confidence

6. Broadcast Variables

Some constraints are common to all agents. Instead of sending the same data repeatedly, Spark can broadcast it.

```
constraints = sc.broadcast(global_constraints)
```

Examples include:

- Maximum budget
- Resource capacity
- Delivery restrictions

- Common business rules

This reduces network communication.

7. Accumulators

Accumulators can be used for simple global statistics.

For example:

```
violations = sc.accumulator(0)
```

The accumulator can count the number of constraint violations.

However, important optimization results should not depend on accumulators because Spark can retry tasks. The actual objective should be calculated from the returned RDD results.

8. Communication Between Agents

Agents can communicate using Kafka or gRPC.

A typical message contains:

Agent ID

Iteration Number

Proposed Solution

Objective Value

Constraint Status

Confidence

Timestamp

Spark handles bulk data processing, while a separate communication system can handle agent coordination.

9. Consensus Mechanism

The local solutions are evaluated and combined.

Local Solutions

↓

Evaluation

↓

Constraint Checking

↓

Conflict Detection

↓

Consensus

↓

Global Solution

↓

Next Iteration

A solution with a better objective value and fewer constraint violations can receive a higher priority.

10. RDD Transformation Sequence

The main RDD sequence is:

Input Problem



parallelize()



map()



mapPartitions()



filter()



reduceByKey()



Evaluation



Consensus



Next Iteration

filter() removes invalid solutions.

mapPartitions() performs local optimization.

reduceByKey() can combine results from different partitions.

11. Fault Tolerance

Optimization is normally iterative, so losing intermediate results can waste a lot of processing time.

Important RDDs should be checkpointed.

optimized.checkpoint()

The checkpoint should be stored on reliable distributed storage.

If an executor fails:

Executor Failure



Lost Partition



RDD Lineage / Checkpoint



Partition Recomputed



Worker Restarts



Optimization Continues

Agent states should also be periodically saved.

12. Scalability

The system can be scaled by:

- Increasing RDD partitions.
- Adding Spark workers.
- Using dynamic allocation.
- Broadcasting common constraints.
- Reducing unnecessary shuffles.
- Using multiple coordinator levels.

For a very large optimization problem:

Global Coordinator

|

Regional Coordinators

|

Worker Agents

This reduces the load on a single coordinator.

13. RDD Persistence

RDDs that are reused during several optimization iterations should be persisted.

Recommended choices:

MEMORY_ONLY

MEMORY_AND_DISK

MEMORY_AND_DISK_SER

MEMORY_AND_DISK is useful when the dataset is larger than available executor memory.

14. Serialization

Agent state should use simple serializable objects.

Avoid sending:

- Database connections
- Open files
- Network sockets
- Large unnecessary objects

between the driver and executors.

Shared read-only information should be broadcast instead of serialized separately for every task.

15. Monitoring

The following should be monitored:

- Number of active agents
- Iteration time

- Objective value
- Constraint violations
- Executor failures
- CPU and memory usage
- Shuffle size
- Communication delays
- Number of failed tasks

Dashboards can be created using Prometheus and Grafana.

16. Security

Communication between agents should be encrypted using TLS. Authentication should be applied to agent services.

Only authorized agents should be allowed to modify optimization states.

Optimization data stored in cloud or distributed storage should be encrypted at rest.

17. Benchmarking

The system can be tested using increasing problem sizes:

1 million states

5 million states

10 million states

50 million states

Example targets:

Candidate evaluations : ≥ 5 million/minute

Iteration latency : ≤ 10 seconds

Recovery time : ≤ 60 seconds

Worker utilization : $\geq 70\%$

Speedup can be measured using:

Speedup = $T1 / TN$

where $T1$ is the execution time with one worker and TN is the execution time with N workers.

18. Conclusion

The multi-agent optimization system divides a large problem into smaller RDD partitions and assigns them to worker agents. Coordinator agents manage the overall process, worker agents solve local problems, and evaluator agents compare the results. Broadcast variables reduce communication overhead, while checkpointing and RDD lineage provide fault tolerance. The architecture can be applied to supply-chain optimization, scheduling, portfolio optimization, and other large-scale decision problems.

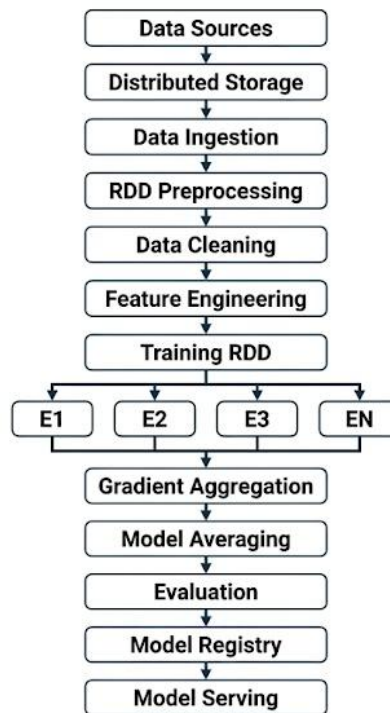
SYSTEM 3 – COMPLETE BIG DATA AI PIPELINE

1. Introduction

A complete AI system needs more than just model training. Large datasets must first be cleaned, transformed, and converted into useful features before they can be given to a machine-learning algorithm.

This system uses PySpark RDDs to build an end-to-end pipeline covering data preprocessing, feature engineering, distributed training, model evaluation, and deployment.

2. Architecture



3. Data Preprocessing

Raw data may contain:

- Missing values
- Duplicate records
- Incorrect formats
- Invalid values
- Outliers

The first step is to convert the raw input into a standard format.

```
parsed = raw_rdd.map(parse_record)
```

Invalid records are removed using:

```
valid = parsed.filter(is_valid)
```

Duplicate records can be removed using:

```
unique_data = valid.distinct()
```

`distinct()` involves a shuffle, so it should be used only when duplicate removal is actually required.

4. Missing Value Imputation

Missing values can be handled using:

- Mean
- Median
- Mode
- Forward filling for time-series data
- Model-based imputation

For numerical data, statistics can be calculated in parallel and then applied to the relevant records.

5. Outlier Detection

Outliers can be detected using:

- Z-score
- IQR
- Domain-specific thresholds

For IQR:

$$\text{IQR} = Q3 - Q1$$

Records that fall far outside the normal range can be removed or marked for further analysis.

6. Feature Engineering

The cleaned data is transformed into features.

The feature-engineering process is:

Clean Data



Encoding



Binning



Feature Crossing



Joining



Final Feature RDD

Feature Crossing

Features can be combined to provide additional information.

For example:

Region + Product Category

Age + Product Category

Device + Location

Binning

Continuous values can be converted into groups.

For example:

0–18

19–30

31–50

51+

7. Join and Cogroup Operations

Suppose customer information and transaction information are stored separately.

They can be combined using:

```
combined = customer_rdd.join(transaction_rdd)
```

When information from multiple datasets needs to be grouped by the same key, `cogroup()` can be used.

```
combined = customer_rdd.cogroup(  
    transaction_rdd,  
    interaction_rdd  
)
```

However, joins and cogroups can create large shuffle operations, so partitioning should be planned carefully.

8. RDD Operation Sequence

The main sequence is:

Raw Data

↓

`map()`

↓

`filter()`

↓

`distinct()`

↓

`map()`

↓

`join()`

↓

`cogroup()`

↓

Feature RDD

↓

`mapPartitions()`

↓

`reduceByKey()`

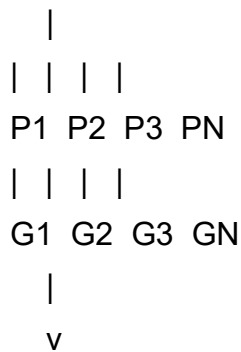
↓

Model Training

9. Distributed Model Training

The training data is divided into partitions.

Training RDD



Gradient Aggregation



Global Parameters



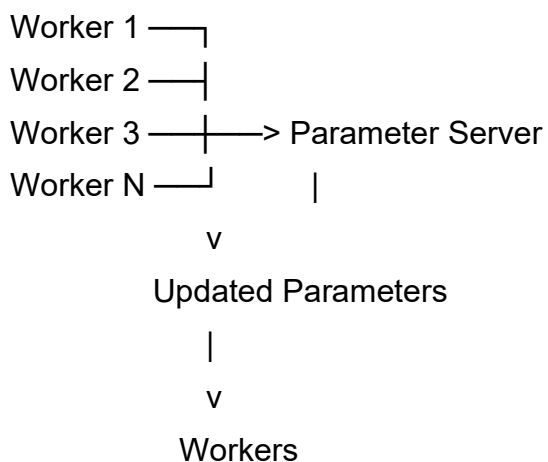
Next Iteration

Each executor calculates a local gradient.

The local gradients are then combined to calculate the global update.

10. Parameter Server

For large models, a parameter server can maintain the current model parameters.



The workers receive the latest model parameters and continue training.

11. Model Averaging

Another approach is local model training followed by model averaging.

Model 1

Model 2

Model 3

...

Model N

The models can be combined to form a global model.

Global Model =

$(\text{Model1} + \text{Model2} + \dots + \text{ModelN}) / N$

Weighted averaging can be used when different partitions contain different amounts of data.

12. Fault Tolerance

RDD lineage is one of Spark's major advantages. If a partition is lost, Spark can reconstruct it by repeating the operations that created it.

For long-running training jobs, checkpointing should be used.

Training



Iteration 1



Checkpoint



Iteration 2



Checkpoint



Iteration 3

Speculative execution can also be enabled to reduce the effect of slow or straggling tasks.

13. Graceful Degradation

If an executor fails:

Executor Failure



Lost Partition



Lineage / Checkpoint



Partition Recomputed



Training Continues

The entire training process should not have to restart because of a single executor failure.

14. RDD Persistence

Frequently reused RDDs should be persisted.

For example:

`training_rdd.persist()`

Possible storage levels include:

MEMORY_ONLY

MEMORY_AND_DISK

MEMORY_AND_DISK_SER

Large datasets generally benefit from MEMORY_AND_DISK because data that cannot fit in memory can be stored on disk.

15. Scalability

The pipeline can scale through:

- Proper RDD partitioning
- Increasing executor nodes
- Dynamic resource allocation
- Broadcast variables
- Efficient joins
- Reducing shuffle operations
- Executor memory tuning
- Data locality

Stage boundaries should be analyzed because shuffle operations create separate Spark stages and can become performance bottlenecks.

16. Serialization

Large Python objects can create serialization overhead.

Therefore:

- Keep data structures compact.
- Use broadcast variables for shared read-only data.
- Avoid repeatedly transferring models.
- Use serializable agent and model classes.
- Avoid storing active connections inside objects sent to workers.

17. Monitoring

The pipeline should monitor:

- Job execution time
- Stage execution time
- Task failures
- Shuffle read/write
- Executor memory
- CPU usage
- Garbage collection
- Training loss
- Model accuracy
- Data quality

Prometheus and Grafana can provide dashboards for these metrics.

18. Security

Data should be encrypted during transmission using TLS and encrypted while stored.

Access should be controlled using role-based permissions.

Sensitive data should be masked or removed when it is not required for training.

19. Benchmarking

The pipeline should be tested using increasing datasets:

10 GB

100 GB

1 TB

10 TB

Example performance targets:

Processing throughput : ≥ 1 million records/sec

P95 stage latency : ≤ 5 minutes/TB

Worker utilization : $\geq 70\%$

Failure recovery : ≤ 2 minutes

Scalability can be measured using:

Speedup = T_1 / T_N

Efficiency = Speedup / N

where N is the number of workers.

20. Conclusion

The complete Big Data AI pipeline uses PySpark RDDs throughout data preprocessing, feature engineering, and distributed model training. Operations such as `map()`, `filter()`, `distinct()`, `join()`, `cogroup()`, and `reduceByKey()` allow the data to be processed in parallel. Distributed gradient computation and model averaging reduce training time. RDD lineage, checkpointing, and speculative execution improve reliability. With proper partitioning, memory management, security, and monitoring, the pipeline can process very large datasets efficiently.

SYSTEM 4 – REAL-TIME DECISION-MAKING FOR CRITICAL INFRASTRUCTURE

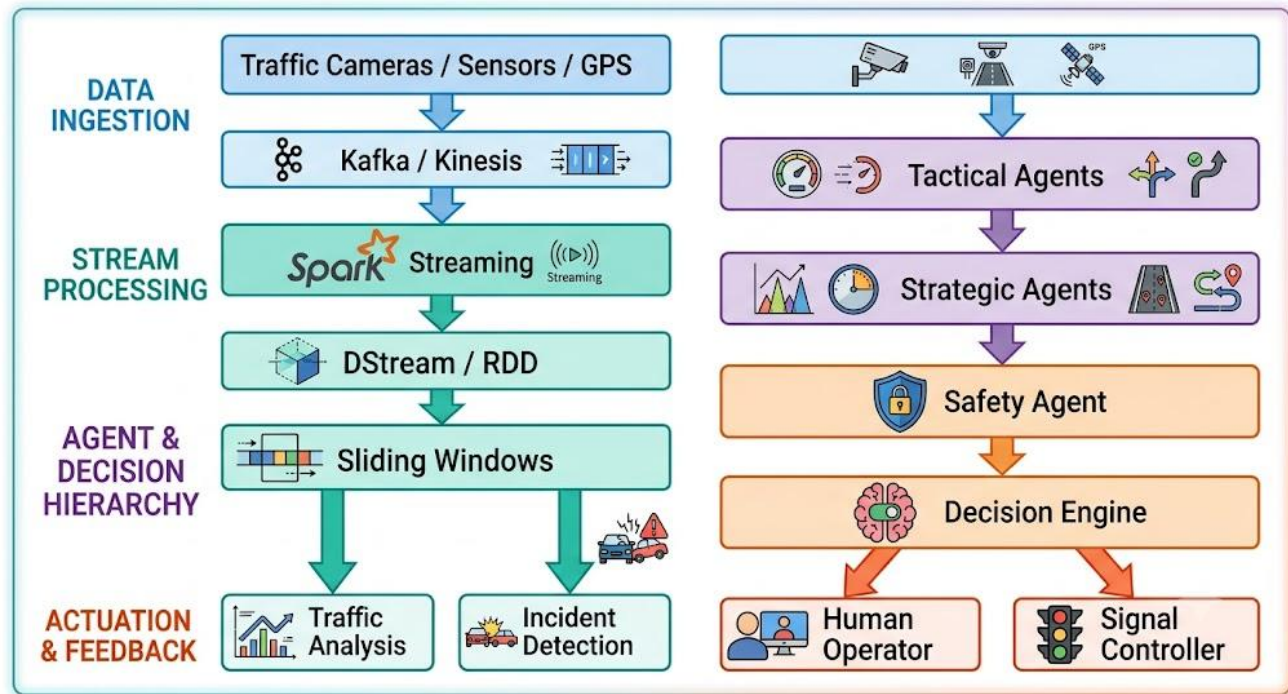
Smart Traffic Management Using PySpark and Intelligent Agents

1. Introduction

Traffic management produces continuous data from cameras, road sensors, GPS devices, and traffic signals. This information needs to be processed quickly to detect congestion and incidents.

This system uses PySpark streaming, RDD operations, sliding windows, and intelligent agents to support real-time traffic decisions. Since traffic control can affect public safety, the system includes safety rules and human supervision.

2. System Architecture



3. Data Sources

The system receives:

- Vehicle Count
- Average Speed
- Road Location
- Lane Occupancy
- Signal Status
- GPS Information
- Incident Reports
- Timestamp

The data is continuously sent to Kafka or Kinesis.

4. Streaming and RDD Processing

DStreams divide the incoming data into small batches.

The basic operation sequence is:

Traffic Event



DStream



RDD



filter()



map()



Sliding Window



reduceByKey()



Traffic Statistics



Incident Detection

Invalid sensor readings are removed using filter().

map() converts events into a standard format.

reduceByKey() can calculate the total vehicle count for each road.

Example:

```
traffic = window_rdd.map(  
    lambda x: (x.road_id, x.vehicle_count)  
)reduceByKey(  
    lambda a, b: a + b  
)
```

5. Sliding Window Processing

Traffic conditions change continuously, so recent data is more useful than very old data.

For example:

Window Size : 60 seconds

Sliding Period : 10 seconds

The system repeatedly analyzes the latest 60 seconds of traffic data every 10 seconds.

This helps identify sudden congestion and unusual changes.

6. Incident Detection

The system compares current traffic conditions with expected conditions.

Possible indicators include:

- Sudden reduction in speed

- Large increase in vehicle count
- Abnormal lane occupancy
- Sudden traffic stoppage
- Sensor inconsistencies

An AI model can calculate an incident probability.

7. Intelligent Agent Hierarchy

Tactical Agent

The tactical agent handles immediate traffic problems.

It may:

- Suggest temporary signal changes.
- Identify alternate routes.
- Alert traffic operators.
- Give priority to emergency situations.

Strategic Agent

The strategic agent studies long-term patterns.

It can analyze:

- Daily congestion
- Peak traffic periods
- Signal coordination
- Road capacity
- Recurring incidents

Safety Agent

The safety agent checks every proposed action before it is implemented.

It verifies:

Minimum green time

Minimum pedestrian crossing time

Emergency vehicle priority

Maximum signal-change frequency

Manual override

Safety rules have higher priority than optimization.

8. Human-in-the-Loop

High-risk decisions should involve a human operator.

AI Recommendation



Safety Validation



Traffic Operator



Approve / Reject



Signal Controller

This reduces the possibility of an incorrect autonomous action.

Low-risk and previously approved adjustments can be automated under controlled conditions.

9. Latency Requirements

The system should respond quickly.

Example targets:

Sensor-to-alert latency : ≤ 2 seconds

Incident detection P95 : ≤ 1 second

Decision P95 : ≤ 500 ms

Availability : $\geq 99.99\%$

These values should be adjusted after testing with the actual traffic infrastructure.

10. Explainability

Every AI recommendation should include a simple explanation.

Example:

Recommendation:

Increase green time by 10 seconds.

Reason:

Vehicle count increased by 35% and average speed decreased by 22% during the recent monitoring period.

Safety Status:

Passed.

The system should record the input values, model version, agent decision, safety checks, and final action.

11. Rollback Mechanism

An incorrect signal adjustment should be reversible.

Abnormal Result



Safety Agent



Rollback Trigger



Previous Safe Configuration



Human Notification

Previous safe configurations should be stored so that they can be restored quickly.

12. Fault Tolerance

The system uses:

- Kafka replication
- RDD lineage
- Checkpointing
- Task retry
- Executor recovery
- Replicated configuration
- Backup traffic-control services

If one Spark worker fails, the lost RDD partition can be recomputed.

13. RDD Persistence

Frequently used traffic statistics can be cached using:

MEMORY_ONLY

MEMORY_AND_DISK

For large traffic datasets, MEMORY_AND_DISK is safer because the system can continue if the RDD does not completely fit in memory.

Critical streaming state should also be checkpointed to reliable distributed storage.

14. Scalability

The system can scale by:

- Increasing Kafka partitions.
- Increasing Spark executors.
- Dividing roads into RDD partitions.
- Processing different geographical regions independently.
- Using dynamic resource allocation.
- Reducing unnecessary shuffle operations.

For example:

City

|

+--- North Zone → Spark Workers

|

+--- South Zone → Spark Workers

|

+--- East Zone → Spark Workers

|

+--- West Zone → Spark Workers

15. Serialization

Traffic events should use compact serializable objects.

Agent classes should contain only required state and configuration. Database connections or open sockets should not be serialized and sent to Spark workers.

16. Monitoring

The system should monitor:

- Sensor input rate
- Processing latency
- Executor health
- Traffic prediction accuracy
- Incident detection accuracy
- Agent decisions
- Safety-rule violations
- Signal-controller status

Alerts should be generated when latency or safety thresholds are exceeded.

17. Security

Traffic sensor and control communication should be encrypted.

Authentication should be required before an agent can access the signal-control system.

The system should use role-based access control so that monitoring agents cannot directly modify signal configurations.

All important decisions should be logged for auditing.

18. Benchmarking

The system can be tested using different traffic loads:

10,000 events/sec

25,000 events/sec

50,000 events/sec

100,000 events/sec

Important measurements include:

- Throughput
- P95 latency
- P99 latency
- Incident-detection accuracy
- Recovery time
- CPU and memory usage

19. Conclusion

The smart traffic management system combines real-time Spark processing with intelligent agents to monitor and respond to changing traffic conditions. Sliding-window RDD processing helps identify recent traffic patterns. Tactical agents handle immediate problems, while strategic agents work on long-term optimization. A safety agent and human operator provide additional protection against

incorrect decisions. Fault tolerance, security, explainability, and rollback mechanisms make the architecture suitable for critical infrastructure applications.

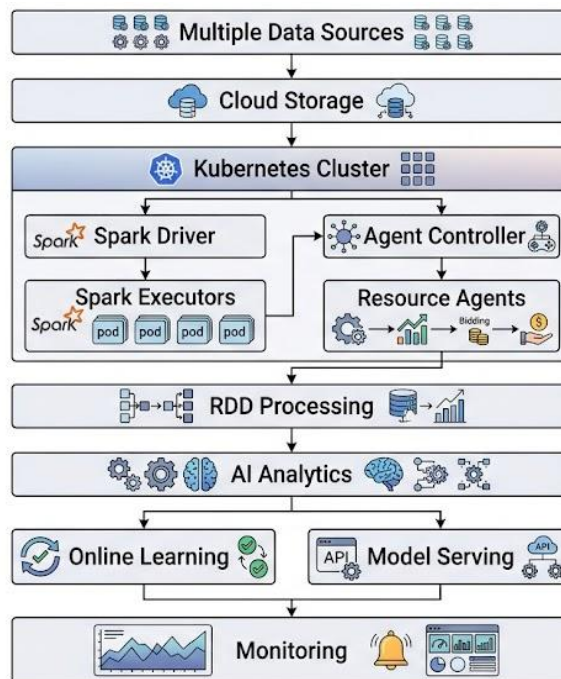
SYSTEM 5 – CLOUD-NATIVE ADAPTIVE AI ANALYTICS PLATFORM

1. Introduction

Cloud platforms provide the flexibility to increase or decrease computing resources according to workload. An adaptive AI analytics platform can use this flexibility to process large datasets while controlling cost and maintaining performance.

This system combines PySpark, RDD processing, Kubernetes, intelligent agents, cloud storage, auto-scaling, and online learning.

2. System Architecture



3. Cloud Infrastructure

The system can be deployed using Kubernetes.

Main components include:

- Kubernetes cluster
- Spark driver
- Spark executor pods
- Cloud object storage
- Model registry
- Identity and access management

Kubernetes allows Spark components to be deployed as containers and makes scaling easier.

4. RDD Processing

The main data flow is:

Cloud Data



RDD



filter()



map()



flatMap()



reduceByKey()



join()



AI Analysis

RDDs can be divided into partitions so that different executors process different parts of the data.

5. Dynamic Resource Allocation

The platform continuously monitors:

CPU Usage

Memory Usage

Input Data Rate

RDD Queue

Processing Latency

Executor Utilization

If workload increases:

High Workload



Monitoring Agent



Scaling Decision



Add Executors



Higher Processing Capacity

When workload decreases, unnecessary executors can be removed.

6. Intelligent Resource Agents

A resource agent monitors the current workload and decides how many resources are required.

A bidding agent can compare:

Cost

Performance

Deadline

Availability

Instance Type

For example:

Tenant Requirement



Bidding Agent



Available Resources



Cost/Performance Comparison



Resource Allocation

The final allocation should still be checked by a central resource controller.

7. Spot Instances

Spot or preemptible cloud instances can be used for workloads that can tolerate interruption.

Suitable workloads include:

- Batch processing
- Data preprocessing
- Experimental model training
- Recomputable Spark jobs

Critical production services should not depend entirely on spot instances.

8. Multi-Tenant Data Isolation

The platform can support multiple organizations.

A simple logical arrangement is:

Tenant A → Partition Group A

Tenant B → Partition Group B

Tenant C → Partition Group C

However, RDD partitioning alone is not enough for security.

Proper isolation should also use:

- IAM

- Separate storage locations
- Network policies
- Access control
- Encryption keys
- Application-level authorization

9. Online Learning

The system can update models continuously instead of retraining them completely.

Existing Model

```
|
+-----+
```

```
|
```

New Data ----+

```
|
```

```
v
```

Incremental Learning

```
|
```

```
v
```

Updated Model

```
|
```

```
v
```

Validation

```
|
```

```
v
```

Model Registry

A new model can first operate in shadow mode. If it performs well, it can gradually replace the existing production model.

10. RDD Persistence

Frequently reused datasets should be persisted.

Suitable levels include:

MEMORY_ONLY

MEMORY_AND_DISK

MEMORY_AND_DISK_SER

Large analytical datasets are usually safer with MEMORY_AND_DISK.

Checkpointing should be used for long RDD lineage chains and important iterative computations.

11. Serialization

Cloud-based Spark applications may have many executors, so serialization efficiency is important.

The system should:

- Keep agent objects small.

- Use broadcast variables for common read-only information.
- Avoid sending large models repeatedly.
- Use serializable custom classes.
- Avoid including active connections inside serialized objects.

12. Data Governance

The platform should maintain:

- Data ownership
- Data classification
- Access permissions
- Data retention periods
- Data lineage
- Model lineage
- Audit logs
- Deletion procedures

Only the minimum required personal or sensitive information should be processed.

13. Cross-Border Data Processing

If data is processed in different countries or regions, the platform must consider applicable privacy and data-residency requirements.

A safer design is:

Region A Raw Data

↓

Regional Processing

↓

Approved Aggregated Data

↓

Global Analytics

This reduces the need to transfer raw personal information across borders.

14. Disaster Recovery

The platform should maintain replicated data and important configuration in another availability zone or region.

Primary Region

|

+---- Replicated Storage

|

v

Secondary Region

The following should be backed up:

- Model registry

- Checkpoints
- Configuration
- Agent state
- Metadata
- Important datasets

Two important disaster-recovery measures are:

RPO – Maximum acceptable data loss

RTO – Maximum acceptable recovery time

An example target is:

RPO $\leq$ 15 minutes

RTO $\leq$ 60 minutes

15. Fault Tolerance

Fault tolerance is achieved through:

- RDD lineage
- Checkpointing
- Executor replacement
- Kubernetes restart policies
- Replicated storage
- Spark task retry
- Speculative execution

If one executor fails, Spark can recompute the lost RDD partition instead of restarting the complete job.

16. Scalability

The system supports both vertical and horizontal scaling.

Horizontal Scaling

Add more Spark executors.

Vertical Scaling

Give an executor more CPU or memory.

Elastic Scaling

Automatically change the number of executors according to workload.

Kubernetes and Spark dynamic allocation can work together to provide this capability.

17. Monitoring and Observability

The system should monitor:

- Kubernetes health
- Spark jobs
- Executor status
- CPU
- Memory

- Network
- RDD storage
- Processing latency
- Cloud cost
- Model accuracy
- Model drift
- Agent decisions

Prometheus and Grafana can be used to display real-time dashboards.

Cloud logs should also be centralized for troubleshooting.

18. Security

Data should be encrypted:

During transmission → TLS

At rest → Storage Encryption

Identity and access management should control who can access each tenant's data.

Network policies should prevent unauthorized communication between containers.

Agent permissions should follow the principle of least privilege.

19. Benchmarking

The platform should be tested using different workloads:

100 GB

1 TB

10 TB

100 TB

Resource configurations can be compared using:

10 Executors

25 Executors

50 Executors

100 Executors

Measurements should include:

- Throughput
- P95 latency
- Cost per workload
- CPU utilization
- Memory utilization
- Scaling time
- Failure recovery time

Example targets:

Interactive analytics P95 : ≤ 2 seconds

Batch throughput : ≥ 1 million records/sec

Executor utilization : $\geq 70\%$

RTO : ≤ 60 minutes

20. Adaptive Resource Optimization

The resource-management agent can compare current performance with the required SLA.

For example:

Latency > SLA



Increase Resources



Check Cost



Measure New Performance



Keep / Roll Back

If resources are being underused:

Low Utilization



Resource Agent



Reduce Executors



Lower Cost

This makes the platform more cost-efficient.

21. Conclusion

The cloud-native adaptive AI analytics platform combines PySpark RDD processing with Kubernetes and intelligent resource-management agents. It can automatically adjust resources based on workload, support multiple tenants, and continuously update AI models. RDD persistence, checkpointing, cloud storage replication, and executor recovery improve reliability. Security, governance, cross-border data controls, monitoring, and disaster recovery are included so that the platform can be used for large-scale cloud AI applications.